🤝 Open Source and Developer Communities

Fulllion Creative Works

Commands and features were checked against GitHub Docs, the GitHub CLI manual, Git's documentation, each community's own pages and each AI tool's help pages in November 2026. Each video's summary was also rechecked against what the video actually says.

Throughout, replace:

Placeholder	With
`<owner>`	the account that owns the original project
`<repo>`	the project's repository name
`<your-username>`	your GitHub username
`<branch-name>`	a short name for your change, such as `fix-readme-typo`

Contents

1. Why Contribute to Open Source
2. Five Videos to Start With
3. Before You Start
4. Find a Project and an Issue
5. Read the Project's Rules
6. Your First Contribution, Step by Step
7. Etiquette and What Happens Next
8. Developer Communities
9. How to Ask a Good Question
10. Search and AI Coding Tools
    - Give the assistant context first
    - Read an unfamiliar repository faster
11. Using AI Tools Responsibly
    - Check your data controls
12. Common Mistakes
13. Sources

---

1. Why Contribute to Open Source

Open source projects publish their code so anyone can read it, use it and improve it, under the terms of a license. Contributing to one teaches things that solo projects can't:

- **Reading other people's code.** Most professional work is changing code someone else wrote.
- **Working the way teams work:** issues, branches, pull requests, code review and automated tests.
- **Public proof of your skills.** A merged pull request is something an employer can click on and read.
- **Connections.** Maintainers and other contributors are working developers.

You don't have to write code to contribute. Fixing documentation, improving error messages, adding tests, translating, reporting bugs clearly and answering other people's questions all count, and they're often the best first step.

2. Five Videos to Start With

Each of these takes a slightly different angle. Watch one or two before you start, and the rest when you want more depth.

Video	Creator	Best for
Contributing to Open Source Can Change Your Life - Here's How to Do It	Learn Fast Make Things	The quickest overview: a three-minute, seven-step walkthrough from finding a project to waiting for review
How To Get Started With Open Source	Web Dev Simplified	A 13-minute demo of a whole pull request (fork, clone, branch, commit, push, open it), then the same pull request from the maintainer's side as they comment on it and merge it
How to Get Started with Contributing to Open Source	AssemblyAI	Judging whether a project is worth your time, and finding something to work on
Complete Guide to Open Source - How to Contribute	freeCodeCamp.org (taught by Eddie Jaoude)	A 42-minute course: a tour of GitHub, finding projects, writing good issues and pull requests in Markdown, and maintaining your own projects
How to contribute to open source	strager	A real contribution, start to finish: fixing a bug in the GDB debugger and sending the fix by email, in about 45 minutes

The rest of this guide turns their advice into steps, checked against GitHub's own documentation.

Jump to, in the freeCodeCamp course: why open source matters · a tour of a GitHub repository · issues · a pull request from the GitHub website · finding projects · Markdown for issues and pull requests · small and draft pull requests · GitHub Sponsors · what he looks for when hiring · maintaining your own project · your GitHub profile

> **What's changed since the freeCodeCamp course:** it calls Codespaces and Discussions upcoming beta features; both are now standard parts of GitHub, and personal accounts get a free monthly Codespaces allowance. It also says GitHub matches sponsorships. That matching fund has closed. GitHub still charges no fee on sponsorships from personal accounts, and up to 6% on those from organizations.

3. Before You Start

You need three things:

1. **A GitHub account,** with a profile that has your name or handle and a short bio.
2. **Git, installed and set up** with your name and email. The CLI Cheat Sheet has the install command, and Publishing to GitHub with gh shows how to commit under the right name and email.
3. **Optional: the GitHub CLI (`gh`),** which can fork, clone and open pull requests from the terminal. See Install the GitHub CLI and Sign In.

If you've never made a pull request, practice first with **First Contributions**. It's a repository built for exactly this: you add your name to a file and open a pull request, with step-by-step instructions for every operating system.

4. Find a Project and an Issue

Start with what you use

The best first project is one you already use, such as a library, a tool, a VS Code extension or a documentation site. You already know what it does, and you'll care about the result. Learn Fast Make Things suggests looking at your own project's dependency list (`package.json` in a JavaScript project, or `go.mod` in Go): every package there is a candidate. Web Dev Simplified adds that **larger projects** can be easier to start with, because a bigger community means more people to help you. If you make games, the same applies: the Godot engine and many game libraries are open source, and Learn Game Development compares the engines.

Look for beginner labels

Many projects mark beginner-friendly issues with labels. GitHub's documentation highlights two:

- `good first issue`: small, well-described tasks for newcomers.
- `help wanted`: tasks the maintainers would welcome help with.

Ways to find them:

1. **On a project you like:** open its **Issues** tab and filter by the `good first issue` label.
2. **The contribute page:** add `/contribute` to a repository's address, such as `https://github.com/<owner>/<repo>/contribute`, to see its beginner-friendly issues.
3. **GitHub Topics:** browse projects by subject, such as `https://github.com/topics/javascript`.
4. **GitHub Explore:** recommendations based on what you've starred and worked on.
5. **Good First Issue:** a list of beginner issues from popular projects, run by DeepSource.
6. **First Timers Only**, started by Scott Hanselman and Kent C. Dodds: a friendly starting point that collects tutorials and issue-finding sites, and explains the `first-timers-only` label some projects use for issues reserved for people who have never contributed before.

7. **Up For Grabs** and **CodeTriage**, both suggested by AssemblyAI. Up For Grabs lists projects that set aside tasks for new contributors. CodeTriage sends you an open issue from projects you pick, so you can help a little at a time.
8. **Search a project for `TODO`.** Developers leave these notes for later and often never open an issue for them. Both AssemblyAI and the freeCodeCamp course suggest it: find one, open an issue about it, and offer to fix it.
9. **Fix the docs.** Typos, unclear sentences and missing steps are real contributions. As the freeCodeCamp course points out, a newcomer is often the best person to notice a step the documentation skips.

Check the project is active and welcoming

Before you spend time on a project, check these (from GitHub's Open Source Guides):

- [ ] It has a **LICENSE** file. Without one, it isn't really open source.
- [ ] The latest commit is recent, and commits are regular.
- [ ] Issues get responses, and some get closed.
- [ ] Pull requests are reviewed and merged within a reasonable time.
- [ ] Maintainers are polite and helpful in issues and pull requests, and thank contributors.
- [ ] **Closed** issues and pull requests that weren't accepted got an explanation, not just a silent close.

AssemblyAI and the freeCodeCamp course both recommend checking this, because it shows how your work will be treated if it isn't merged.

5. Read the Project's Rules

Before you change anything, read these files in the repository. They're usually linked from the main page.

File	What it tells you
README	What the project does, and how to install and run it
CONTRIBUTING (`CONTRIBUTING.md`)	How the maintainers want issues and pull requests written, how to set up the project, and how to run its tests. It can live in the root, `docs` or `.github` folder, and GitHub links to it when you open an issue or pull request
CODE_OF_CONDUCT	How people are expected to treat each other
LICENSE	What you, and everyone else, may do with the code, including your contribution
Issue and pull request templates	The details the maintainers want. Fill them in, don't delete them

If CONTRIBUTING says to open an issue before a pull request, or to sign a Contributor License Agreement (CLA), do that first.

6. Your First Contribution, Step by Step

These steps follow GitHub's Contributing to a project guide. The commands work in PowerShell, Git Bash, macOS and Linux.

6.1 Claim the issue

1. Comment on the issue to say you'd like to work on it, and briefly how you plan to fix it. This stops two people doing the same work.
2. If the issue is old, ask whether it's still needed.
3. Wait for a maintainer to reply, if the project's CONTRIBUTING file asks you to.

6.2 Fork and clone

A **fork** is your own copy of the project on GitHub. You can't push to the original, but you can push to your fork and ask the original to pull in your changes.

1. On the project's GitHub page, click **Fork**, then **Create fork**. Leave **Copy the DEFAULT branch only** ticked.
2. Clone your fork (not the original) to your computer:

   ```
   git clone https://github.com/<your-username>/<repo>.git
   cd <repo>
   ```

3. Add the original project as a second remote called `upstream`, so you can pull in its updates later:

   ```
   git remote add upstream https://github.com/<owner>/<repo>.git
   git remote -v
   ```

   You should see `origin` (your fork) and `upstream` (the original).

With the GitHub CLI instead, one command forks and clones:

```
gh repo fork <owner>/<repo> --clone
cd <repo>
git remote -v
```

Check the remotes with `git remote -v`. If there's no `upstream`, add it with step 3.

For a small text fix, you can skip the command line. As the freeCodeCamp course shows, open the file on GitHub and click the pencil (**Edit**) icon. GitHub asks you to **Fork this repository**, then you make the change, click **Propose changes**, add a title and description, and click **Create pull request**. It's ideal for a typo in a README, but for code you'll want to run the tests locally first.

6.3 Create a branch

Make every change on its own branch, never on `main`. One branch means one pull request.

```
git switch -c <branch-name>
```

This creates the branch and switches to it. (Older guides use `git checkout -b <branch-name>`, which does the same.)

6.4 Make and test your change

1. Set the project up the way its README or CONTRIBUTING file describes, and make sure it runs **before** you change anything.
2. Make your change. Keep it small and focused on the issue. Don't reformat unrelated code.
3. Run the project's tests and linter, if it has them, and check your change works.

6.5 Commit and push

1. Check exactly what you've changed:

   ```
   git status
   git diff
   ```

2. Stage the files you meant to change, and commit with a clear message:

   ```
   git add <file>
   git commit -m "Fix typo in installation steps"
   ```

   Follow the project's commit message style if CONTRIBUTING describes one.
3. Push the branch to your fork:

   ```
   git push -u origin <branch-name>
   ```

6.6 Open the pull request

1. Open your fork on GitHub. A banner offers to **Compare & pull request**. Click it (or click **Contribute**, then **Open pull request**).
2. Check that the pull request goes **from** your branch **to** the original project's main branch.
3. Write a title that says what the change does, and a description that says **why**, how you tested it, and which issue it fixes. Writing `Fixes #123` in the description links the pull request to issue 123, and GitHub closes the issue when the pull request is merged.
4. Add screenshots for anything visual.
5. Click **Create pull request**. If you want early feedback on unfinished work, choose **Create draft pull request** instead.

With the GitHub CLI instead, from your branch:

```
gh pr create --web
```

This opens the pull request form in your browser, already filled in from your branch.

6.7 Keep your fork up to date

The original project keeps changing. To bring your fork's `main` up to date, either click **Sync fork**, then **Update branch** on your fork's GitHub page, or run:

```
git fetch upstream
git switch main
git merge upstream/main
git push origin main
```

To bring those updates into a branch you're still working on:

```
git switch <branch-name>
git merge main
```

If Git reports a **conflict**, open the files it lists, choose which lines to keep, then `git add` and `git commit` them. The CLI Cheat Sheet has more Git commands for branches and undoing mistakes.

6.8 Projects that don't use pull requests

Some long-running projects take changes by email instead of GitHub pull requests. strager's video shows one, the GDB debugger, from start to finish:

1. **Build the project from source** following its README, installing whatever the build says is missing, and check you can reproduce the bug.
2. **Search the code** for the functions involved, and try a quick hard-coded fix to prove you've found the right place.
3. **Turn the fix into a proper change.** His became a new user setting, made by copying how an existing setting was written.
4. **Read the contributor guide.** For GDB it asked for a commit message explaining the reasoning and the testing, plus updates to the manual and the project's `NEWS` file for any new command.
5. **Send the patch to the project's mailing list.** GDB's maintainers point contributors to its Contribution Checklist and the gdb-patches list. strager attached his patch, made with `git format-patch`; regulars on the list have also asked newcomers to use `git send-email`, which puts the patch inline in the email so reviewers can reply to it line by line.

The maintainers replied with suggestions, and the whole thing took him about 45 minutes. If a project's CONTRIBUTING file or website mentions a mailing list or "patches", follow its instructions rather than opening a pull request.

7. Etiquette and What Happens Next

While you work

- Keep discussion in public issues and pull requests, not private messages. The exception is a security problem, which should go through the project's security policy (often a `SECURITY.md` file).
- Search existing issues, the README and the docs before asking. Say what you've already tried.
- Be brief and specific. Maintainers are often volunteers.

Make it easy to review (tips from the freeCodeCamp course)

- **Keep pull requests small.** Eddie Jaoude, who maintains projects himself, would rather merge several small ones than spend a day on one huge one.
- **Want early feedback?** Open a **draft** pull request, with a task list (`- [ ]` and `- [x]`) in the description showing what's done. Mark it ready for review when it's finished.
- **Write** `Closes #123` (or `Fixes #123`) in the description, so the issue closes when your pull request is merged.
- **Use Markdown well:** code in fenced blocks with the language named, a `diff` block to show a before and after, and quotes (`>`) to make clear which part of a comment you're replying to.
- **When reviewing,** you can suggest an exact change that the author applies with one click. The course describes this as a diff; GitHub's name for it is a **suggestion** block, added with the **Suggestion** icon in a review comment.

After you open the pull request

- **Automated checks** may run. If one fails, open it, read the error, and push a fix to the same branch. The pull request updates itself.
- **Changes requested?** That's normal, not a rejection. Reply to each comment, push the fixes, and say thanks. If you can't finish, say so, so someone else can pick it up.
- **No reply?** Maintainers have already been notified, and many work on the project in their spare time, so don't send them direct messages about it. Wait about a week, then leave one polite comment asking if anyone has time to review. If there's still nothing, it's fine to move on to another project.
- **Not accepted?** Politely ask for feedback, so you understand why. Don't argue. The project may have plans you don't know about.

Things to avoid

- Opening many tiny pull requests just to raise your contribution count. Maintainers notice, and some projects ban it.
- Submitting AI-generated code or text you haven't read, tested and understood. Some projects have their own rules about AI-assisted contributions, so check CONTRIBUTING.
- Taking over an issue someone else has claimed, unless they've gone quiet and the maintainers agree.

You're a maintainer too. The freeCodeCamp course points out that every public repository you own is an open source project someone could contribute to. To make yours welcoming, open its **Insights** tab and click **Community Standards**: GitHub's checklist shows whether it has a README, code of conduct, contributing guide, license and issue templates. Add a quick-start section and a screenshot to the README, automate checks with GitHub Actions, and reply to new issues and pull requests promptly, even if only to say when you'll look properly. Eddie Jaoude, the course's teacher, says that when he hires he looks less at how polished someone's code is and more at whether it's improving over time and how they communicate in issues and reviews.

8. Developer Communities

Communities are where you ask for help, see what other developers are talking about, and later, help others. Lurk for a while first to get a feel for each one.

Reddit

Reddit communities each have their own rules, shown in the sidebar (on mobile, under **About**). The summaries below are from the communities' published guidelines. Read the current rules before posting.

Subreddit	What it's for	Know before you post
r/learnprogramming	Questions from people learning to code, at any level	Its posting guidelines ask you to say what you've tried, share a small, formatted example of your code, and include the output you expected, what you got, and the full error. Title-only posts are removed automatically. Read the FAQ first, since many questions are already answered there
r/programming	Links and discussion about programming, for working developers	Not a help forum. "How do I...?" questions, debugging requests and very beginner-level articles are removed, and career posts must be about software engineering careers. Bring help questions to r/learnprogramming
r/ProgrammingLanguages	The theory, design and building of programming languages	For people interested in how languages work or making their own. Questions about learning a particular language fit better in r/learnprogramming

DEV Community

DEV is a social network and blogging site for developers, built on the open source Forem software. It's a good place to write about what you're learning (which also helps your portfolio), read beginner-friendly articles, and join **DEV Challenges**. Like any open blogging site, article quality varies, so check dates and test code before you rely on it.

Stack Overflow

Stack Overflow is a question and answer site, not a discussion forum. Its huge archive of answered questions is often the fastest fix for an error message: search it first. If you ask a new question, follow Section 9 closely, because unclear or duplicate questions get closed.

Two of its rules to know:

- **Generative AI content is banned.** You may not use AI tools to write questions or answers there. Content found to be AI-generated is likely to be deleted, and repeat posting can lead to a suspension.
- **Questions must be specific, practical programming problems,** written in English. Mostly opinion-based questions can be closed, and requests to recommend a book, tool, library or tutorial are off-topic.

GeeksforGeeks

GeeksforGeeks has a huge free library of articles on data structures, algorithms, languages and interview questions. It's handy for a quick explanation or another angle on a topic. **Use it as a starting point, not the final word:** quality varies between articles, and some are outdated or contain mistakes. Check anything important against official documentation, such as MDN for the web or the Python docs.

Discord servers and forums

Discord servers give you real-time help and a sense of company while you learn. Some well-run ones:

- **The Odin Project**: a large, friendly community for anyone following its curriculum. Its guides on what to try before asking and how to ask a technical question are worth reading even if you never join.
- **Python Discord**: one of the biggest Python communities, with dedicated help channels and volunteer Helpers. Read its rules.
- **Reactiflux**: a chat community of tens of thousands of React and JavaScript developers. Read its Code of Conduct.
- **freeCodeCamp forum**: not Discord, but a busy, beginner-friendly forum for anyone following freeCodeCamp.

Staying safe on Discord: don't run files or code that strangers send you, never share your Discord token or any password, and be wary of private messages offering jobs, paid help or crypto. Ask in public channels instead.

9. How to Ask a Good Question

This works on every site above. It follows Stack Overflow's How do I ask a good question? and minimal reproducible example guides.

1. **Search and research first.** Keep a note of what you found and why it didn't help, and mention it in your question.
2. **Write a title that sums up the specific problem.** "Why does my fetch call return undefined in React?" beats "React problem, please help".
3. **Explain the problem in words before showing code.** What are you trying to do, and what's stopping you?
4. **Include a minimal, reproducible example:**
   - **Minimal:** as little code as possible that still shows the problem.
   - **Complete:** everything someone needs to run it, pasted into the question.
   - **Reproducible:** test it yourself first to make sure it really shows the problem.
5. **Paste code and errors as text, never screenshots,** formatted as code. Include the exact error message and which line causes it.
6. **Say what you expected and what happened instead.** "It doesn't work" isn't enough.
7. **Add the right tags or channel,** such as the language and framework.

8. **Proofread, then stay around** to answer follow-up questions. When someone solves it, say thanks and mark the answer.

> **Watch for the XY problem.** Ask about the problem you're actually trying to solve, not only the workaround you're stuck on. See xyproblem.info.

10. Search and AI Coding Tools

The original notes recommended two search tools from 2022 and 2023. Both have changed:

Tool	What it was	Status in November 2026
Grepper (codegrepper.com)	A browser extension that showed saved code answers in your search results	**Don't use it.** codegrepper.com now redirects to You.com, an AI search company, and users of the browser extension report that it no longer works. If you still have the extension installed, remove it
Phind (phind.com)	An AI search engine for developers	**Shut down.** Phind closed in January 2026, with mid-January reported as its last day of search

GitHub Copilot

GitHub Copilot is GitHub's AI coding assistant. It works in VS Code and other editors, on GitHub.com, and in the terminal. In November 2026, GitHub's documentation lists these main features:

- **Inline suggestions:** Copilot suggests code as you type, and you choose whether to accept it.
- **Chat:** ask questions about your code, a repository or a programming concept, and ask it to explain, change, refactor or fix code.
- **An agent:** give it a task, and it researches the repository, makes the changes and prepares a pull request for you to review.
- **Code review:** it can review pull requests and suggest improvements.

Plans (from GitHub's plans page):

- **Copilot Free:** a limited monthly allowance, including 2,000 inline suggestions a month.
- **Copilot Student:** free for verified students.
- **Copilot Pro:** $10 a month. Verified teachers and maintainers of popular open source projects may get it free.
- **Pro+ and Max:** higher tiers with more usage and more models.

What happened to Copilot Labs? The original notes linked GitHub's short How to translate code into other languages: GitHub Copilot Labs. Copilot Labs was an experimental extension from 2022, and it was retired at the end of 2023 when its features moved into Copilot Chat. To translate code between languages now, select it and ask Chat, for example "Rewrite this function in Python." Then check the result as carefully as you'd check your own code.

Give the assistant context first

An AI assistant only knows what you tell it. Ikaneisha Jenkins makes this the first step in The First Thing You Should Do Before Using ChatGPT for Business. Her point is that most people don't need better prompts, they need to give the tool better context. Her method, shown in ChatGPT for a pretend bakery:

1. **Ask the AI to interview you** about your business, your audience, what you'll use it for and how you communicate. Let it ask as many questions as it wants, or set a limit. Rough, honest answers are fine.
2. **Answer in batches** (say, questions 1 to 10), because it may change its later questions based on your earlier answers.
3. **Ask for a one-paragraph summary** of what it learned, and paste it into the "about you" part of **Personalization** (her June 2026 video shows a **More about you** field). Turn memory on.
4. **Ask for a second interview,** of up to 10 questions, about **how you want it to respond**, then paste the summary of that into **Custom instructions**.
5. **Test it** with a real task, and tell it plainly when something doesn't sound right.

Her video is about business, but it works just as well for coding: ask it to interview you about your operating system, editor, languages, experience and how you like things explained. Saved instructions stop you repeating "I'm a beginner, I use Windows, explain each step" in every chat. A result might look like this:

```
I'm learning web development on Windows, with VS Code, Node.js and Python.
Explain new terms briefly, and go one step at a time.
Say when you're not sure, and link the official documentation where you can.
Before suggesting a package, tell me how to check it's the real one.
```

Each tool has two related features. **Instructions** are what you write yourself, and they apply to every chat. **Memory** is what the tool learns from your chats over time. Settings were checked in November 2026, and menus move, so look for the same names if yours differ:

Tool	Your instructions	Memory
ChatGPT	**Settings** ▸ **Personalization**, with **Enable customization** on, then the **Custom Instructions** field (on iOS and Android, **Settings** ▸ **Customize ChatGPT**). Free and Go plans allow 1,500 characters, paid plans 5,000	**Settings** ▸ **Personalization** ▸ **Memory**. It can use saved memories and, where offered, **Reference chat history**. You can review, correct or delete what it remembers
Claude	**Settings** ▸ **Your Claude** ▸ **Instructions** (called **Instructions for Claude** under **Account** in the older layout). **Projects** also have their own project instructions	On by default on Free, Pro and Max. Turn it off with **Generate memory from chats** in **Settings** ▸ **Your Claude**, and select **Manage** next to **Your memory** to review it. Each project has its own memory, and incognito chats (the ghost icon) aren't saved to memory or history
GitHub Copilot	In Copilot Chat on GitHub.com, select your profile picture, then **Personal instructions**. For a whole repository, add a `.github/copilot-instructions.md` file, which Copilot uses for requests in that repository	**Copilot Memory**, on paid plans, remembers facts about a repository and your coding preferences. Your preferences are on by default for Pro, Pro+ and Max. Memories expire after 90 days by default

Keep instructions short and about how you work, not private details. A `.github/copilot-instructions.md` file is part of the repository, so on a public project everyone can read it. Check your memory now and then, because it can be wrong or out of date, and see **Check your data controls** (Section 11) before you share anything sensitive.

Read an unfamiliar repository faster

Opening a big project for the first time is hard. AriaCodez shared these shortcuts in a short Facebook video, which demonstrates GitDiagram, Gitingest and the full-stop key; the original notes also listed DeepWiki and GitMCP from the same post. Each was checked against its own documentation.

On GitHub itself: press `.` (full stop) on any repository or pull request page to open it in **github.dev**, a free VS Code editor that runs in your browser. Press `>` to open it in a new tab instead, or change `github.com` to `github.dev` in the address. You need to be signed in. It's for reading, searching and small edits: there's no terminal, and you can't run the code. You can install extensions, but only those built to run on the web appear.

Third-party tools: each one works by changing part of a public repository's address.

Tool	Change the address to	What you get
DeepWiki	`deepwiki.com/<owner>/<repo>`	An AI-written wiki of the project, with architecture diagrams and links to the source, that you can ask questions. From the team behind the Devin coding agent
GitDiagram	`gitdiagram.com/<owner>/<repo>` (change `hub` to `diagram`)	An interactive architecture diagram. Click a part to jump to its files. Open source (MIT)
Gitingest	`gitingest.com/<owner>/<repo>` (change `hub` to `ingest`)	The whole project as one text file, with the folder tree, file contents and a token count, ready to paste into an AI chat. Open source (MIT)
GitMCP	`gitmcp.io/<owner>/<repo>`	An MCP server address you add to an AI coding assistant, so it can read the project's documentation and code when it needs to. Open source (Apache 2.0)

> **Before you use them:** these are separate services, not GitHub. The address trick works on **public** repositories, and using it sends the project's code to that service, and often on to an AI model. GitDiagram and Gitingest can read a private repository if you give them a GitHub token, but that hands your code and a key to a third party, so don't do it with work, client or school code unless you're allowed. AI-written summaries and diagrams can be wrong, so check anything important against the code itself.

11. Using AI Tools Responsibly

AI tools can make you faster, or they can stop you from learning. These habits keep them useful.

Understand code before you use it

GitHub's own best practices say Copilot can make mistakes and you "should always validate the code it suggests." Before you accept anything:

1. **Read it, and be able to explain every line.** If you can't, ask the tool to explain it, then check that explanation against the documentation.
2. **Run it and test it,** including edge cases. AI code often looks right and is subtly wrong, or uses functions and packages that don't exist.
3. **Check it for security problems,** such as unchecked user input, SQL built from strings, or secrets written into the code.
4. **Check any package it suggests** really exists and is the one you meant, before you install it. The Safer Package Installs guide explains why.

Check licenses

- **Code without a license isn't free to use.** As choosealicense.com explains, no license generally means you have no permission to use, change or share it.
- **AI suggestions can match public code.** Copilot can show you when a suggestion matches code in a public GitHub repository, along with that repository's license, or you can block matching suggestions in your Copilot settings. See Finding public code that matches GitHub Copilot suggestions.
- **Follow each community's rules.** Stack Overflow bans AI-written content, and some open source projects restrict AI-assisted contributions.

Never paste secrets

Don't paste passwords, API keys, tokens, `.env` files, customer data or private company code into an AI tool, a forum or a chat server. Replace them with placeholders like `<your-api-key>` first. GitHub's **push protection** is on by default for your account and blocks many known secret formats from being pushed to public repositories, but it can't catch everything. If a secret leaks, revoke it and make a new one straight away. Keep Secrets Out of Your Repository covers `.gitignore`, `.env` files and scanning before you push.

Check your data controls

What you type into an AI tool is stored, can be reviewed for safety, and on some plans is used to train future models. Before you use one for real work:

1. **Never paste secrets, client data or other people's private information** (above). Turning training off doesn't change that.
2. **Check the training setting** for each tool you use:
   - **ChatGPT** (Free, Plus and Pro): **Improve the model for everyone** is on by default. Turn it off in **Settings ▸ Data controls**. A **temporary chat** isn't used for training, isn't saved in your history and doesn't update memory, though OpenAI may keep it for up to 30 days for safety. Rating a response with thumbs up or down can still share that whole conversation for training.

    - **Claude** (Free, Pro and Max, including Claude Code on those plans): **Settings** ▸ **Privacy** ▸ **Help Improve our AI models**. Turning it off stops new chats and coding sessions being used in future training.
    - **GitHub Copilot** (Free, Pro, Pro+ and Max): since 24 April 2026, GitHub can use your Copilot data for AI model training. To opt out, click your profile picture ▸ **Copilot settings**, and set **Allow GitHub to use my data for AI model training** to **Disabled**.
3. **Using a work or school account?** Your organization controls these settings, and its AI policy comes first. OpenAI, for example, doesn't train on ChatGPT Business, Enterprise or Edu workspaces by default.
4. **Remember these are separate switches.** Turning training off doesn't delete your chats or your memory, and turning memory off doesn't stop training. Delete chats you no longer want kept.

Learning versus delegating

- **While you're learning, write it yourself first.** Use AI to explain errors, review your attempt, or suggest what to read next, not to hand you the finished answer. Struggling with a problem is how the skill forms.
- **Use AI as a study aid, not a substitute.** Learning How to Learn Programming covers techniques, such as explaining a topic in your own words and testing yourself from memory, that AI can support but can't do for you.
- **Delegate what you already understand.** Boilerplate, repetitive edits and first drafts of tests are good jobs to hand over, because you can spot when the result is wrong.
- **Practice without it sometimes.** Most technical interviews don't allow AI help (Coding Practice and Interview Prep), and you'll need to debug the code you ship either way.

12. Common Mistakes

What you see	Why	What to do
`Permission denied` or `403` when you push	You cloned the original project instead of your fork	Fork it, then point `origin` at your fork: `git remote rename origin upstream`, then `git remote add origin https://github.com/<your-username>/<repo>.git`
Your pull request includes changes you didn't make	Your branch started from an old or different branch, or you worked on `main`	Update `main` (Section 6.7), create a new branch from it, and copy your change across
This branch has conflicts that must be resolved	The original project changed the same lines you did	Merge `main` into your branch (Section 6.7), fix the conflicts, commit and push
A check on your pull request fails	A test, linter or build step failed	Click **Details** on the failed check, read the error, fix it on the same branch and push again
Your question was closed or removed	It broke the community's rules, or it was a duplicate or unclear	Read the rules and the moderator's note, improve the question (Section 9), or ask in a better-suited community

What you see	Why	What to do
You pushed a password or key	It was in a file you committed	Revoke the key and create a new one now. Removing it in a new commit isn't enough, because it stays in the history

13. Sources

Contributing (GitHub and Git documentation)

- GitHub Docs: Contributing to a project
- GitHub Docs: Finding ways to contribute to open source on GitHub
- GitHub Docs: Setting guidelines for repository contributors
- GitHub Docs: Configuring a remote repository for a fork
- GitHub Docs: Syncing a fork
- GitHub Docs: Linking a pull request to an issue
- GitHub CLI manual: gh repo fork and gh pr create
- Git documentation: git switch
- Open Source Guides: How to Contribute to Open Source
- First Contributions
- Good First Issue
- Up For Grabs
- CodeTriage
- freeCodeCamp: How to Contribute to Open Source (resource list)
- GitHub Docs: Editing files
- GitHub Docs: Commenting on a pull request (suggested changes)
- GitHub Docs: Accessing a project's community profile
- GitHub Docs: About GitHub Sponsors
- GitHub Docs: About billing for GitHub Codespaces
- GDB wiki: Contribution Checklist and the gdb-patches mailing list archive

Communities

- r/learnprogramming, r/programming and r/ProgrammingLanguages
- DEV Community: About
- Stack Overflow: What topics can I ask about here?
- Stack Overflow: How do I ask a good question?
- Stack Overflow: How to create a Minimal, Reproducible Example
- Stack Overflow: Policy on generative AI content
- GeeksforGeeks
- The Odin Project: community guides
- Python Discord rules

- Reactiflux
- freeCodeCamp forum

AI tools

- GitHub Docs: About GitHub Copilot
- GitHub Docs: Copilot plans
- GitHub Docs: Best practices for using GitHub Copilot
- GitHub Docs: Finding public code that matches Copilot suggestions
- GitHub Docs: Push protection
- choosealicense.com: No license
- OpenAI Help: ChatGPT Custom Instructions
- OpenAI Help: Memory in ChatGPT
- OpenAI Help: Data controls in ChatGPT
- Claude Help: Understanding Claude's personalization features
- Claude Help: Use Claude's chat search and memory to build on previous context
- Anthropic Privacy Center: How do I change my model improvement privacy settings?
- GitHub Docs: Custom instructions for GitHub Copilot
- GitHub Docs: Adding personal custom instructions for GitHub Copilot
- GitHub Docs: About GitHub Copilot Memory
- GitHub Docs: Managing GitHub Copilot policies as an individual subscriber

Reading a repository

- GitHub Docs: The github.dev web-based editor
- DeepWiki documentation (Devin docs)
- GitDiagram (GitHub repository)
- Gitingest (GitHub repository)
- GitMCP (GitHub repository)

Videos

- Web Dev Simplified, How To Get Started With Open Source
- strager, How to contribute to open source
- AssemblyAI, How to Get Started with Contributing to Open Source
- freeCodeCamp.org (Eddie Jaoude), Complete Guide to Open Source - How to Contribute
- Learn Fast Make Things, Contributing to Open Source Can Change Your Life - Here's How to Do It
- GitHub, How to translate code into other languages: GitHub Copilot Labs
- Ikaneisha Jenkins, The First Thing You Should Do Before Using ChatGPT for Business
- AriaCodez, a short video on GitHub repository shortcuts (credited by name only; it's on Facebook, which needs a login)

Latest version: github.com/ImmuneMoon/Open-Source-and-Developer-Communities

If this helped, you can support Fulllion Creative Works through PayPal or Ko-fi.

© 2026 Fulllion Creative Works. All rights reserved.